

# ECE 6140 Project 1-1 Report

## Circuit Simulation

Jingtian Dang

October 2, 2025

### 1. Data Structures Used

The simulator was implemented using a modular, object-oriented approach with the following core data structures:

- **Gate (gate.py):** Represents a logic element (AND, OR, NAND, NOR, XOR, etc., plus INPUT/OUTPUT). Each gate stores its name, type, input values, output value, and fan-in/fan-out connections. It provides methods to evaluate logic, propagate signals, reset state, and manage connections.
- **GateValue (gate.py):** Implements a five-valued logic system:  $\{0, 1, X \text{ (unknown)}, D, \overline{D}\}$ . It supports logical operations ( $\wedge$ ,  $\vee$ ,  $\oplus$ , inversion) following D-algebra rules for fault simulation.
- **Wire (circuit.py):** Models the connection between one driving gate and multiple output gates. Each wire keeps track of its source gate and destination gates, facilitating fan-in/fan-out linking.
- **Circuit (circuit.py):** Encapsulates the overall digital circuit. It stores all gates and wires, manages primary inputs/outputs, and provides methods for evaluation (via BFS propagation), reset, and visualization.
- **CircuitWrapper (circuit.py):** Parses a netlist (.txt) file to build a **Circuit**. It reads INPUT/OUTPUT declarations and gate definitions, constructs the gates, and connects them. It also provides the `evaluate_circuit_with_normal_input()` function for simulation.

These structures provide modularity, clarity, and extensibility, closely mirroring digital circuit concepts.

### 2. Simulation Algorithm

The simulator evaluates a circuit using breadth-first propagation. The main steps are:

1. Reset all gates to the unknown state.
2. Apply the input vector by setting primary input gate values.
3. Initialize a queue with the fan-out gates of all primary inputs.
4. While the queue is not empty:
  - (a) Dequeue a gate.

- (b) Evaluate its logic function based on fan-in values.
- (c) If the output changes (from UNKNOWN to a known value), enqueue all its fan-out gates.

5. Collect the outputs from all primary output gates.

#### Pseudo-code:

```
function simulate(circuit, input_vector):
    circuit.reset_all_gates()
    for each input_port in circuit.inputs:
        set INPUT gate output to 0 or 1

    queue <- fan-out gates of inputs
    while queue not empty:
        gate <- queue.pop()
        prev <- gate.output
        new <- gate.forward()
        if new != prev:
            assert prev == UNKNOWN
            enqueue fan-outs of gate

    return outputs of primary outputs
```

### 3. Simulation Results

| Circuit        | Test Input               | Output Vector              |
|----------------|--------------------------|----------------------------|
| <b>s27</b>     | 1110101                  | 1001                       |
|                | 0001010                  | 0100                       |
|                | 1010101                  | 1001                       |
|                | 0110111                  | 0001                       |
|                | 1010001                  | 1001                       |
| <b>s298f_2</b> | 10101010101010101        | 00000010101000111000       |
|                | 01011110000000111        | 00000000011000001000       |
|                | 11111000001111000        | 00000000001111010010       |
|                | 11100001110001100        | 00000000100100100101       |
|                | 01111011110000000        | 11111011110000101101       |
| <b>s344f_2</b> | 10101010101010101111111  | 101010101010101010101101   |
|                | 010111100000001110000000 | 00011110000000100001111100 |
|                | 11111000001111000111111  | 00011100000111011000111010 |
|                | 111000011100011000000000 | 00001101111001111111000010 |
|                | 01111011110000000111111  | 10011101111000001001000100 |
| <b>s349f_2</b> | 10101010101010101111111  | 101010101010101011010101   |
|                | 010111100000001110000000 | 00011110000000101011110000 |
|                | 11111000001111000111111  | 00011100000111010001111100 |
|                | 111000011100011000000000 | 00001101111001110010001111 |
|                | 01111011110000000111111  | 10011101111000001010000100 |